Slack Error Intelligence — Implementation Guide

Version 1.0 · August 2026

Companion to ARCHITECTURE.md. That document explains *what the system is and why*. This one explains *how it is built, how to run it, and how to change it*.

Contents

1. Getting started
2. Repository layout
3. Configuration
4. Module walkthrough
5. The pipeline, line by line
6. Working with the database
7. Testing
8. The eval harness
9. Diagnostics
10. Deploying to GitHub Actions
11. Extending the system
12. Troubleshooting

---

1. Getting started

Prerequisites

- Python **3.11 or newer** (`python3 --version`)
- A Slack bot token with `channels:history` (or `groups:history`) and `chat:write`
- An Anthropic API key
- A Voyage AI API key

Install

```
git clone <repo> && cd slack-error-intelligence
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

Every new shell needs `source .venv/bin/activate`. The prompt shows `(.venv)` when it is active.

Verify without any credentials

```
make test     # 63 tests, no network, no secrets
make demo     # full pipeline against fixtures, offline encoder, prints instead of posting
make eval     # scores the pipeline against the script it replaces
```

If those three pass, the codebase is sound. Everything after this point is configuration.

First real run

```
cp .env.example .env      # then fill in the three keys
rei run --days 1 --dry-run
```

`--dry-run` prints the Block Kit payload instead of posting. Read the labels before letting it write to a channel other people watch.

Build the corpus, then go live

```
rei backfill --days 180   # one-time; never posts
rei stats
rei run --days 1          # posts for real
```

---

2. Repository layout

```
slack-error-intelligence/
├── src/rei/                the package
├── tests/                  63 tests, none require network
├── evals/                  metrics + ablation harness
├── fixtures/               labelled synthetic data in the real wire format
├── scripts/
│   ├── generate_fixtures.py regenerate the labelled fixture set
│   ├── inspect_payloads.py  masked diagnostic report over a real dump
│   └── build_docs.py        render these documents to PDF
├── docs/                   architecture + implementation, .md and .pdf
├── .github/workflows/
│   ├── daily.yml            the scheduled run
│   └── ci.yml               lint, types, tests, eval gate
├── data/                   local SQLite corpus (git-ignored)
├── pyproject.toml
├── Makefile
└── .env                    git-ignored; never committed
```

---

3. Configuration

All settings are environment variables prefixed `REI_`, read by `pydantic-settings` from the environment or a git-ignored `.env`.

Variable	Default	Meaning
`REI_SLACK_BOT_TOKEN`	—	Bot token. **Secret.**
`REI_SOURCE_CHANNEL_ID`	`C0123456789`	`#error-alerts`
`REI_DEST_CHANNEL_ID`	`C0987654321`	`#error-digest`
`REI_MENTION_USER_IDS`	`["U0123456789"]`	JSON list, no angle brackets
`REI_ANTHROPIC_API_KEY`	—	**Secret.**
`REI_ANTHROPIC_MODEL`	`claude-sonnet-5`	Swap to `claude-haiku-4-5` for lower cost
`REI_VOYAGE_API_KEY`	—	**Secret.**
`REI_VOYAGE_MODEL`	`voyage-3-lite`	
`REI_CLUSTER_THRESHOLD`	`0.90`	τ for the Voyage encoder
`REI_OFFLINE_CLUSTER_THRESHOLD`	`0.76`	τ for the offline encoder
`REI_EMBEDDING_BATCH_SIZE`	`96`	Reduced automatically under free-tier throttling
`REI_LOOKBACK_DAYS`	`1`	Daily window
`REI_ENABLE_DIAGNOSIS`	`false`	Adds root cause + suggested fix to the report
`REI_MAX_QUOTES_INLINE`	`5`	Quote IDs shown before "…more in thread"
`REI_MAX_CLUSTERS_IN_MESSAGE`	`12`	Issues in the channel view
`REI_DB_PATH`	`data/rei.db`	Corpus location
`REI_DRY_RUN`	`false`	Print the payload, post nothing
`REI_OFFLINE`	`false`	No network at all — local encoder, rule labeller

Secrets are typed `SecretStr`, so they do not appear in logs or tracebacks.

---

4. Module walkthrough

`config.py`

One `Settings` class. Add a field here and it is immediately available as `REI_<NAME>`, typed and validated.

`models.py`

`ErrorEvent`, `Cluster`, `ClusterReport`, `DailyReport` — pydantic models. `ErrorEvent` is the important one: it carries `quote_id` **and** `quote_id_confidence` separately from `related_ids`, which is the fix for the old script's quote inflation.

`http.py`

One `HttpClient` used by all three providers. Retries on `{408, 409, 425, 429, 500, 502, 503, 504}` with exponential backoff and jitter, honouring `Retry-After` when present. Raises `ApiError` carrying provider, status and body — which is how `embeddings.py` detects Voyage's free tier from the response text.

`slack.py`

Paged history with adaptive pacing, thread harvesting, posting. Detects the restricted rate-limit tier by observing that a page came back much smaller than requested, and slows to one request per minute rather than colliding with 429s.

`extract.py`

The biggest module and the one with the most hard-won behaviour. Three input shapes are tried in order: labelled rich-text blocks (the real format), JSON payloads, and a loose-text regex fallback. See ARCHITECTURE §6.

Key functions:

Function	Does
`is_reportable`	Filters policy documents, system events, non-error payloads
`labelled_pairs`	Extracts label/value pairs; parses both `blocks` and `text`, keeping whichever recovered more
`parse_error_value`	Splits the `error` value's sub-fields; tolerant of newline *and* space separation, recovers from truncated JSON
`labelled_meta`	Pulls `methodName` and `correlationId` so they survive whichever parser wins
`find_quote_id`	Contextual quote matching with a confidence score

`masking.py`

`normalize()` is the entry point: strip embedded JSON → mask volatile tokens → collapse whitespace → bound length. `is_low_information()` decides whether a template needs enrichment.

`embeddings.py`

Cache-first. `Embedder.embed()` takes `{hash: text}`, returns `{hash: vector}`, and only calls the API for hashes not already stored. `offline_embed()` is a deterministic hashed character-n-gram encoder used when no key is present — a test double, never a production path.

`store.py`

All SQL. Additive migrations run on open. `search_clusters()` is the one function to change if you ever outgrow exact search.

`clustering.py`

`IncrementalClusterer.assign()` returns one `Assignment` per template, flagging `is_new_cluster` and `is_new_template`. The second flag is what triggers relabelling of an existing cluster whose membership changed.

`rag.py`

`Retriever.neighbours()` turns the clusterer's near-miss list into rendered context: labels, exemplars, all-time counts, recorded human resolutions.

`llm.py`

`Labeller.label_clusters()` with forced tool use. `_fill_gaps()` is the defensive merge — discards anything that is not a dict with an in-range integer index, and guarantees exactly one label per input cluster. `_rule_based_label()` is the deterministic fallback.

`pipeline.py`

Orchestration. `build_templates()` is module-level and stateless so the eval can score that stage alone. All counting happens in `aggregate()`.

`blocks.py`

Block Kit rendering with Slack's limits enforced at construction, not hoped for.

`digest.py`

The historical report: ownership rules, quick-win heuristics, correlation-ID linking, HTML rendering, Slack summary.

`baseline.py`

A faithful port of the original script. Never imported by the pipeline — it is measurement apparatus for the eval.

---

5. The pipeline, line by line

`Pipeline.run()` in `pipeline.py` is ~30 lines and reads top to bottom:

```
def run(self, messages, window_start, window_end) -> DailyReport:
    run_date = window_end.strftime("%Y-%m-%d")
    self.store.start_run(run_date)

    events = extract_events(messages)                        # 1
    templates, counts, examples, by_template = build_templates(events)   # 2
    vectors = self.embedder.embed(templates)                 # 3  (cache-first)
    assignments = self.clusterer.assign(                     # 4
        templates, vectors, counts, seen_at=window_end.isoformat()
    )
    new_count = self.label_new_clusters(assignments, by_template)        # 5
    self.persist(assignments, by_template, examples)         # 6
    report = self.aggregate(                                 # 7  ← all counting
        assignments, by_template, window_start, window_end, len(messages)
    )
    self.store.finish_run(run_date, ...)                     # 8
    return report
```

Step 7 is deliberately last. Steps 1–6 mutate state; step 7 reads it back. No number in the report can be influenced by the model, because the model already ran and its output was already persisted.

What each step costs on a steady day

Step	Cost
1 extract	pure CPU, milliseconds
2 templates	pure CPU
3 embed	**zero API calls** — every template hash is cached
4 cluster	zero API calls — every template already has a cluster
5 label	**zero API calls** — nothing changed
6–8	SQLite writes

The whole run is a Slack fetch plus local work.

---

6. Working with the database

The corpus is a plain SQLite file. Open it with anything.

```
sqlite3 data/rei.db
```

Useful queries

```sql
-- Top issues all time
SELECT cluster_id, label, category, total_events, template_count
FROM clusters ORDER BY total_events DESC LIMIT 20;

-- Daily volume for one cluster
SELECT date(occurred_at) AS day, COUNT(*) AS n
FROM events WHERE cluster_id = 1 GROUP BY day ORDER BY day DESC LIMIT 30;

-- Which quotes a cluster affected
SELECT DISTINCT quote_id FROM events
WHERE cluster_id = 1 AND quote_id IS NOT NULL;

-- Attribution quality
SELECT quote_id_confidence, COUNT(*) FROM events GROUP BY 1 ORDER BY 1 DESC;

-- What a cluster actually contains
SELECT text, event_count FROM templates WHERE cluster_id = 3 ORDER BY event_count DESC;

-- Two clusters sharing a trace — the incident-linking signal
SELECT correlation_id, GROUP_CONCAT(DISTINCT cluster_id) AS clusters
FROM events WHERE correlation_id IS NOT NULL
GROUP BY correlation_id HAVING COUNT(DISTINCT cluster_id) > 1 LIMIT 20;

-- Run telemetry
SELECT * FROM runs ORDER BY run_date DESC LIMIT 14;
```

Schema changes

Add the column to `_SCHEMA` **and** to `Store._migrate()`. The schema script only applies to a fresh file; `_migrate()` is what upgrades an existing corpus in place. An existing corpus represents hours of backfill and must never require a rebuild for an additive change.

```python
def _migrate(self) -> None:
    existing = {row["name"] for row in self.conn.execute("PRAGMA table_info(events)")}
    if "my_new_column" not in existing:
        self.conn.execute("ALTER TABLE events ADD COLUMN my_new_column TEXT")
    self.conn.commit()
```

Index creation for a migrated column must live in `_migrate()`, not `_SCHEMA` — the schema script runs first, before the column exists, and indexing a non-existent column fails the whole open.

Rebuilding

```sh
rm -f data/rei.db && rei backfill --days 180
```

Events are re-derived from Slack, so nothing is lost **except human resolutions** — export the `resolutions` table first if any exist.

Threshold changes do **not** retroactively re-cluster. A rebuild is required.

---

7. Testing

```
make test                                    # everything
pytest tests/test_labelled_format.py -v      # one file
pytest -k correlation -v                     # one topic
```

No test requires network or secrets. `REI_OFFLINE=true` swaps the encoder for the deterministic local one and the labeller for rules.

What the suites cover

File	Covers
`test_masking.py`	Token masking, status-code preservation, generic-message detection
`test_extract.py`	JSON path, quote attribution, message selection
`test_labelled_format.py`	The real wire format: truncated payloads, space-separated sub-fields, handler preservation, document filtering
`test_pipeline.py`	End-to-end, cluster stability, idempotency, counter integrity, Block Kit limits, label robustness, correlation linking, ownership determinism
`test_store_and_metrics.py`	Storage, vector search, and the metrics implementation checked against scikit-learn

Tests that encode a bug that actually happened

These are the ones not to delete:

- `test_correlation_id_is_never_the_quote` — the 162-vs-125 quote inflation
- `test_handler_survives_a_truncated_payload` — Slack dropping the trailing block
- `test_string_results_do_not_crash` — the model returning 149 entries, one a bare string
- `test_counters_do_not_inflate_when_a_window_is_reprocessed` — baseline drift
- `test_category_flip_does_not_change_ownership` — the headline metric moving on a relabel

Writing a new test

Use `rich_message()` from `test_labelled_format.py` to build a payload in the real format:

```
msg = rich_message([
    ("methodName", "renewalHandler"),
    ("correlationId", CORRELATION),
    ("error", "type: payment-failure\nmessage: Card declined"),
    ("message", f"Renewal Flow for {QUOTE} : Card declined"),
])
(event,) = extract_events([msg])
```

8. The eval harness

```
make eval        # score all arms
make sweep       # sweep the clustering threshold
```

Four arms over the same labelled input, each adding one stage, so the improvement is attributable rather than asserted:

1. `baseline (exact string)` — the original algorithm
2. `+ token masking`
3. `+ context enrichment`
4. `+ lexical vectors` — the full pipeline on the offline encoder

A fifth arm scores real Voyage embeddings when `REI_VOYAGE_API_KEY` is set.

Metrics

Implemented dependency-free in `evals/metrics.py` and property-checked against scikit-learn over 25 randomised trials in the test suite.

Metric	Question
Homogeneity	Are unrelated failures merged?
Completeness	Is one failure split across lines?
V-measure	Harmonic mean of the two
Adjusted Rand	Agreement corrected for chance
Attribution precision	Of the quotes claimed, how many are right?

Regenerating fixtures

```
python scripts/generate_fixtures.py
```

Fixtures are synthetic but emitted in the **real wire format**, and every failure shape was observed in a genuine export. If you change the generator, re-run `make eval` — the numbers in ARCHITECTURE §14 and the CI floors both assume the current set.

The CI gate

CI fails the build if homogeneity drops below 0.99, completeness below 0.95, V-measure below 0.97, or if the pipeline scores no better than the baseline it replaces. The eval is a gate, not a report.

9. Diagnostics

When the report looks wrong

```
rei dump-slack --days 7                              # raw messages to fixtures/slack_dump.json
python scripts/inspect_payloads.py fixtures/slack_dump.json
```

Eight sections: coverage, payload shape, service metadata, quote attribution, templates, **genuinely missed messages**, generic messages, and old-vs-new.

All output is masked — UUIDs, emails, Stripe IDs and amounts become placeholders before printing, so it is safe to paste into a ticket or a chat. Only label and key *names* appear raw, and those are schema, not customer data.

Section 6 matters most. For every missed message it prints the labels found, whether `methodName` was recovered, which keys the error block yielded, and the masked head of the value — enough to pinpoint a parser gap without guessing.

Other checks

```
rei stats                     # corpus summary
rei run --days 3 --dry-run    # re-render a wider window without posting
rei tune-threshold --online   # re-sweep τ against real embeddings
```

---

10. Deploying to GitHub Actions

Create the repository

```
cd ~/Claude/slack-error-intelligence
git init && git add -A
git commit -m "Semantic renewal-error clustering with daily Slack report"
gh repo create slack-error-intelligence --private --source=. --push
```

Keep it **private**: `events.raw_excerpt` retains original payloads containing customer emails and business names.

Add secrets and variables

```
gh secret set REI_SLACK_BOT_TOKEN
gh secret set REI_ANTHROPIC_API_KEY
gh secret set REI_VOYAGE_API_KEY

gh variable set REI_SOURCE_CHANNEL_ID --body "C0123456789"
gh variable set REI_DEST_CHANNEL_ID   --body "C0987654321"
gh variable set REI_MENTION_USER_IDS  --body '["U0123456789"]'
```

Seed the state branch

The daily job expects the corpus on an orphan `data` branch. Push the one you already built rather than making CI re-backfill:

```
git checkout --orphan data
git rm -rf . >/dev/null
cp data/rei.db rei.db
git add -f rei.db
git commit -m "state: seed from local backfill"
git push -u origin data
git checkout main
```

If you skip this the workflow bootstraps an empty branch and day one shows every issue as `NEW`.

Verify before the first scheduled fire

Actions tab → **Daily renewal error report** → *Run workflow* → set `dry_run: true`. That exercises secrets, state restore, the pipeline and the state commit without posting to Slack.

The schedule

`0 1 * * *` — 01:00 UTC, 06:30 IST. GitHub cron is always UTC and never observes daylight saving.

---

11. Extending the system

Add a masking rule

`masking.py`, `_PATTERNS`. Order matters — most specific first. Add a test to `test_masking.py` asserting that two concrete values collapse to one template.

Support a new payload shape

`extract.py`. Add a `_from_<shape>()` returning the same field dict, and slot it into the chain in `extract_events()`. Keep `labelled_meta()` merging afterwards so `methodName` and `correlationId` survive regardless of which parser wins.

Change what the model returns

`llm.py`: update `_tool_schema()` and the system prompt together, then handle the new field in `pipeline.label_new_clusters()`. Add it to `_rule_based_label()` too, or the fallback path produces incomplete labels.

Share of total

Both reports show each issue's percentage of all errors in the window alongside the raw count. `ClusterReport.share_pct` (daily) and `Issue.share_pct` (historical) compute it from the window total, which is set once during aggregation. A count without its denominator reads as alarming or trivial depending on what the reader guesses the total was.

Add a report section

`blocks.py` for Slack, `digest.py` for the HTML. Both have limit-aware helpers — use `_section()` and `_context()` rather than building dicts by hand, or the 50-block cap will truncate something you cared about.

Turn on root-cause diagnosis

```
REI_ENABLE_DIAGNOSIS=true rei run --days 1 --dry-run
```

Adds `root_cause` and `suggested_action` to the schema, the prompt and the report. Off by default until label quality has been watched for a fortnight.

Swap the vector index

Only `Store.search_clusters()` changes. Worth doing north of ~10^5 vectors; below that, exact search is faster *and* correct.

Point it at a different channel

Everything is configuration — `REI_SOURCE_CHANNEL_ID` and `REI_DEST_CHANNEL_ID`. A different *service* will need extraction work; run `scripts/inspect_payloads.py` against a dump first to see how far off the format is.

---

12. Troubleshooting

Symptom	Cause	Fix
`not_authed` / `invalid_auth`	Token wrong or not saved	Check `.env`; re-open with `open -e .env`
`not_in_channel`	Bot is not a member	`/invite @your-bot` in the channel
Voyage `429` mentioning a payment method	Free tier: 3 req/min	Automatic — it throttles and continues. Add a card to remove the cap
`No events in the corpus`	Digest before backfill	`rei backfill --days 180`
Everything shows `NEW` every day	State not persisting	Check the `data` branch exists and the workflow's persist step ran
Two report lines look like the same failure	τ too high	Lower `REI_CLUSTER_THRESHOLD`, delete the DB, re-backfill
One line mixes unrelated failures	τ too low	Raise it, delete the DB, re-backfill
Labels are terse and mechanical	No Anthropic key — rule fallback	Set `REI_ANTHROPIC_API_KEY`
Clusters look lexical rather than semantic	No Voyage key — offline encoder	Set `REI_VOYAGE_API_KEY`
Coverage dropped after a service change	Payload format changed	`rei dump-slack` then `scripts/inspect_payloads.py`, read section 6
Counters look inflated	Should be impossible	`test_counters_do_not_inflate_when_a_window_is_reprocessed` guards this; file it as a bug

Reading the logs

```
rei.pipeline: extracted 61 error events from 91 messages   ← coverage
rei.pipeline: 15 distinct templates after masking          ← masking effectiveness
rei.embeddings: 15 requested, 15 cache hits, 0 to compute  ← cache health
rei.pipeline: run complete: 11 clusters (0 new), 0 embed calls, 0 llm calls
```

On a healthy steady day: cache hits equal requests, new clusters is 0, and both API call counts are 0. Anything else means the taxonomy changed — which is sometimes exactly what you want to know.